

Multi-Level Parking Assignment Optimization

Optimization Project

Abstract

This report presents a dynamic multi-level parking assignment optimization model. A real smart-parking dataset is used to estimate demand characteristics, while a synthetic multi-level parking structure is generated to satisfy the project requirement of modeling floors, capacities, space attributes, and walking distances. Four solution approaches are implemented and compared: greedy assignment, genetic algorithm, simulated annealing, and integer linear programming. The implemented constraints include vehicle assignment, time-aware space capacity, size compatibility, EV charging requirements, floor capacity, dynamic arrival/departure behavior, and unassigned-vehicle penalties. Experimental results show that the ILP gives the best offline benchmark solution, while simulated annealing and genetic algorithms improve over the greedy baseline.

1 Introduction

This project studies the assignment of vehicles to parking spaces in a multi-level parking facility. Vehicles arrive over time and must be assigned to available and compatible spaces. The goal is to reduce user inconvenience while satisfying operational constraints such as space capacity, size compatibility, electric vehicle charging requirements, and dynamic arrival/departure behavior.

The inconvenience considered in this project is mainly based on walking distance to the exit, with penalties for vehicles that cannot be assigned. The problem is solved and compared using four methods: a greedy online algorithm, a genetic algorithm, simulated annealing, and an integer linear programming model.

2 Problem Description

The parking facility consists of several floors. Each floor contains a fixed number of parking spaces. Each space has attributes such as size, floor, position, and whether it has an electric vehicle charger.

Each vehicle is characterized by:

- arrival time,
- parking duration,
- departure time,

- vehicle type,
- required size category,
- whether it requires electric charging.

The objective is to assign vehicles to spaces while minimizing the total assignment cost and respecting all feasibility constraints.

3 Project Requirements Coverage

Table 1 summarizes how the implementation matches the requested project requirements.

Table 1: Coverage of Project Requirements

Requirement	Status	Implementation Description
Multi-level parking structure	Included	3 floors with 20 spaces per floor
Parking spaces with attributes	Included	Floor, position, size, EV charger availability
Vehicle set with attributes	Included	Arrival time, duration, type, size need, EV charging need
Walking distance matrix	Included	Synthetic internal distance matrix and exit-distance vector
Vehicle assigned exactly once	Included	Assignment or unassigned dummy variable
One vehicle per space at a time	Included	Time-overlap constraints and dynamic repair procedures
Size compatibility	Included	Compact/SUV/van hierarchy
EV charging requirement	Included	EVs requiring charging must use charger spaces
Floor capacity	Included	Enforced through physical space capacity
Dynamic arrivals/departures	Included	Arrival and duration used by all solvers
Ramp/elevator access capacity	Partial	Stored as a parameter, not enforced as hard constraint
Congestion	Partial	Simplified; reported through peak floor occupancy

4 Dataset and Synthetic Instance Generation

The project uses the recommended smart parking dataset from Kaggle:

<https://www.kaggle.com/datasets/datasetengineer/smart-parking-management-dataset>

The dataset contains information such as vehicle type, electric vehicle status, parking duration, occupancy rate, entry time, spot size, and parking status.

Since the dataset does not include a multi-level parking structure, a synthetic parking infrastructure was generated. The dataset is used to estimate demand characteristics, while the parking structure is generated artificially.

The generated benchmark instance contains:

- multiple parking floors,
- a fixed number of spaces per floor,
- size categories for parking spaces,
- electric vehicle charging spaces,
- synthetic walking distances to the exit,

- generated vehicles with arrival times and durations.

All random generation is controlled using a fixed seed to ensure reproducibility.

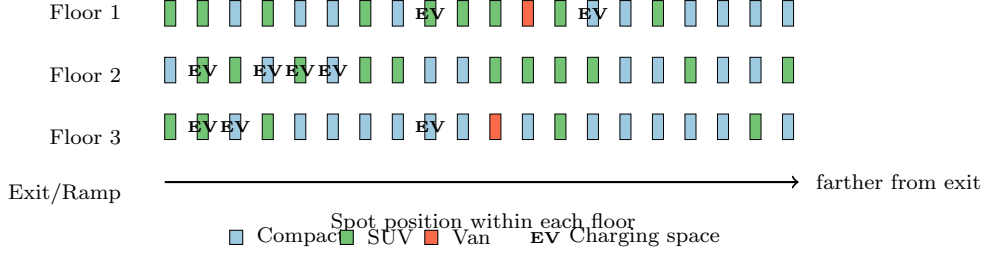


Figure 1: Synthetic multi-level parking structure generated for the benchmark instance. Each row is a floor and each rectangle is a parking space. The colors represent space size categories and the “EV” label indicates a charging-equipped space.

5 Implemented Constraints

This section describes the constraints that are explicitly taken into account in the implementation.

5.1 Vehicle Assignment Constraint

Each vehicle must be assigned to exactly one parking space, or marked as unassigned if no feasible space is available.

Let x_{vs} be a binary variable equal to 1 if vehicle v is assigned to space s , and 0 otherwise. Let y_v be a binary variable equal to 1 if vehicle v is unassigned.

$$\sum_{s \in S} x_{vs} + y_v = 1 \quad \forall v \in V \quad (1)$$

This guarantees that a vehicle cannot be assigned to multiple spaces.

5.2 Parking Space Capacity Over Time

Each parking space can accommodate at most one vehicle at a time. However, since the problem is dynamic, the same parking space can be reused by another vehicle after the first vehicle departs.

For two vehicles v_1 and v_2 , their parking intervals overlap if:

$$\max(a_{v_1}, a_{v_2}) < \min(d_{v_1}, d_{v_2}) \quad (2)$$

where a_v is the arrival time and d_v is the departure time.

If two vehicles overlap in time, they cannot be assigned to the same space:

$$x_{v_1s} + x_{v_2s} \leq 1 \quad \forall s \in S, \forall (v_1, v_2) \text{ overlapping} \quad (3)$$

This is one of the most important constraints because it makes the model time-aware.

5.3 Size Compatibility Constraint

Each vehicle can only be assigned to a space that is large enough. The size hierarchy used is:

$$\text{compact} < \text{suv} < \text{van} \quad (4)$$

Therefore:

- compact vehicles can use compact, SUV, or van spaces,
- SUV vehicles can use SUV or van spaces,
- van vehicles can only use van spaces.

If a vehicle does not fit in a space, that assignment is forbidden:

$$x_{vs} = 0 \quad \text{if vehicle } v \text{ is too large for space } s \quad (5)$$

5.4 Electric Vehicle Charging Constraint

Electric vehicles that require charging must be assigned to parking spaces equipped with chargers.

If vehicle v requires charging and space s does not have a charger, the assignment is forbidden:

$$x_{vs} = 0 \quad \text{if } v \text{ requires charging and } s \text{ has no charger} \quad (6)$$

If no charger-equipped space is available at the required time, the vehicle may be marked as unassigned and receives a penalty in the objective function.

5.5 Floor Capacity Constraint

Each floor has a fixed number of physical parking spaces. Since each physical space can contain at most one vehicle at a time, floor capacity is automatically respected.

For example, if a floor contains 20 spaces, then at most 20 vehicles can be assigned simultaneously to that floor.

5.6 Dynamic Arrival and Departure Constraint

Vehicles arrive and depart dynamically. A space occupied by a vehicle is unavailable until that vehicle's departure time. After departure, the space becomes available again.

This dynamic behavior is considered in:

- the greedy algorithm,
- the genetic algorithm repair procedure,
- the simulated annealing repair procedure,
- the ILP overlap constraints.

5.7 Unassigned Vehicle Penalty

If a vehicle cannot be assigned to any feasible space, it is marked as unassigned and receives a penalty in the objective function. This avoids producing infeasible solutions while still making unassignment undesirable.

$$\text{Penalty Cost} = P \sum_{v \in V} y_v \quad (7)$$

6 Constraints Not Fully Modeled

The project statement also mentions access constraints, such as ramp or elevator capacity. In the current implementation, ramp capacity exists as an instance parameter, but it is not enforced as a hard optimization constraint.

Congestion is also modeled in a simplified way. The greedy solver includes a basic congestion component, and floor occupancy is reported during evaluation. However, a detailed traffic-flow congestion model is left as future work.

Therefore, the implemented constraints are:

- vehicle assignment,
- parking space capacity over time,
- size compatibility,
- EV charging requirement,
- floor capacity,
- dynamic arrival and departure,
- unassigned vehicle penalty.

The partially included or future-work constraints are:

- ramp/elevator access capacity,
- detailed congestion modeling.

7 Objective Function

The objective is to minimize total user inconvenience. In the implemented model, this is represented mainly by walking distance and unassigned vehicle penalties.

$$\min \sum_{v \in V} \sum_{s \in S} c_{vs} x_{vs} + P \sum_{v \in V} y_v \quad (8)$$

where:

- c_{vs} is the cost of assigning vehicle v to space s ,
- P is the penalty for leaving a vehicle unassigned,

- x_{vs} is the vehicle-space assignment variable,
- y_v is the unassigned vehicle variable.

The main component of c_{vs} is the walking distance from the parking space to the exit.

8 Algorithms Implemented

8.1 Greedy Algorithm

The greedy algorithm is an online method. It processes vehicles in arrival order. For each arriving vehicle, it checks all currently available compatible spaces and assigns the best one.

This method is fast and realistic for real-time systems, but it does not have future knowledge.

8.2 Genetic Algorithm

The genetic algorithm is a population-based metaheuristic. Each chromosome represents preferred parking assignments for all vehicles. Since random assignments can be infeasible, a repair procedure is applied to ensure that time, size, and EV constraints are respected.

The genetic algorithm uses:

- an initial population,
- tournament selection,
- crossover,
- mutation,
- elitism,
- repair and evaluation.

8.3 Simulated Annealing

Simulated annealing is a local search method. It starts from an initial solution and repeatedly modifies it. Worse moves can be accepted with a probability that decreases over time. This helps the algorithm escape local minima.

The implemented simulated annealing method starts from the greedy solution, applies random assignment changes, repairs the solution, and keeps the best solution found.

8.4 Integer Linear Programming

The ILP model is an exact offline optimization method. It assumes that all arrivals and departures are known in advance. It provides a benchmark solution for comparing the heuristic methods.

The ILP explicitly includes binary variables, assignment constraints, compatibility constraints, EV charger constraints, and time-overlap constraints.

9 Experimental Results

The experiment was run using the following instance:

- 3 floors,
- 20 spaces per floor,
- 60 total spaces,
- 60 vehicles,
- seed = 42,
- 7 EV vehicles,
- 9 EV-capable spaces.

The obtained results are shown in Table 2.

Table 2: Comparison of Optimization Methods

Solver	Total Cost	Avg Dist. (m)	EV Sat. (%)	Unassigned (%)	Size Viol. (%)	Runtime (s)
Greedy	5254.5	80.6	100.0	1.7	0.0	0.074
Genetic Algorithm	4963.5	75.7	100.0	1.7	0.0	3.094
Simulated Annealing	4868.5	74.0	100.0	1.7	0.0	1.447
ILP	4697.5	71.1	100.0	1.7	0.0	0.107

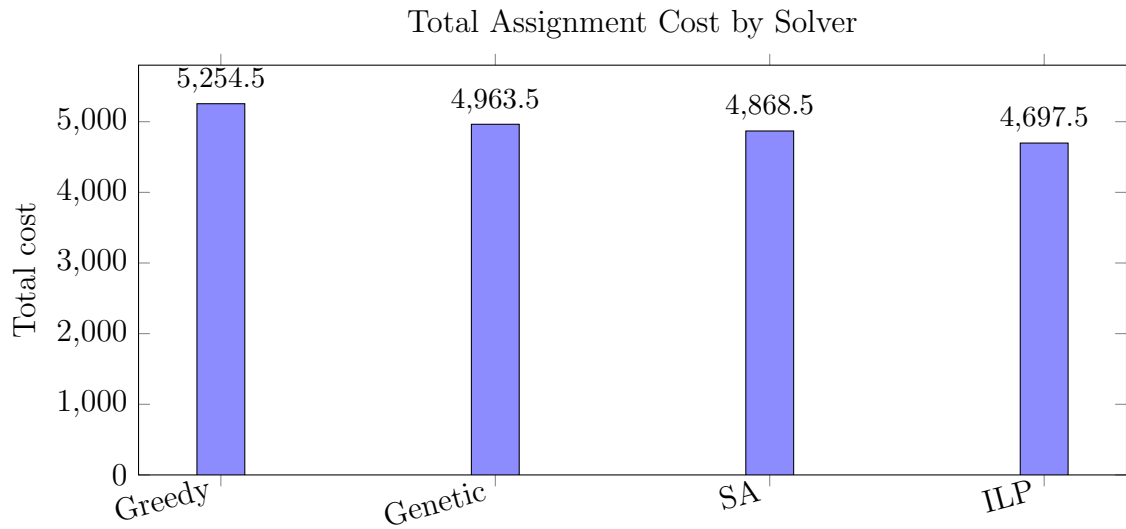


Figure 2: Comparison of total assignment cost. Lower values indicate better assignment quality.

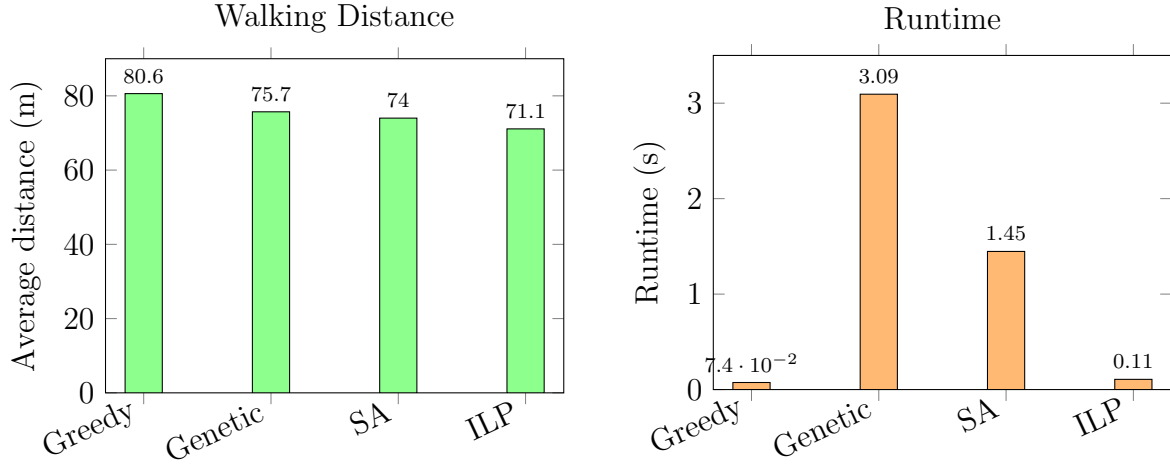


Figure 3: Trade-off between walking distance and runtime. Greedy is fastest, while ILP gives the best distance for this instance.

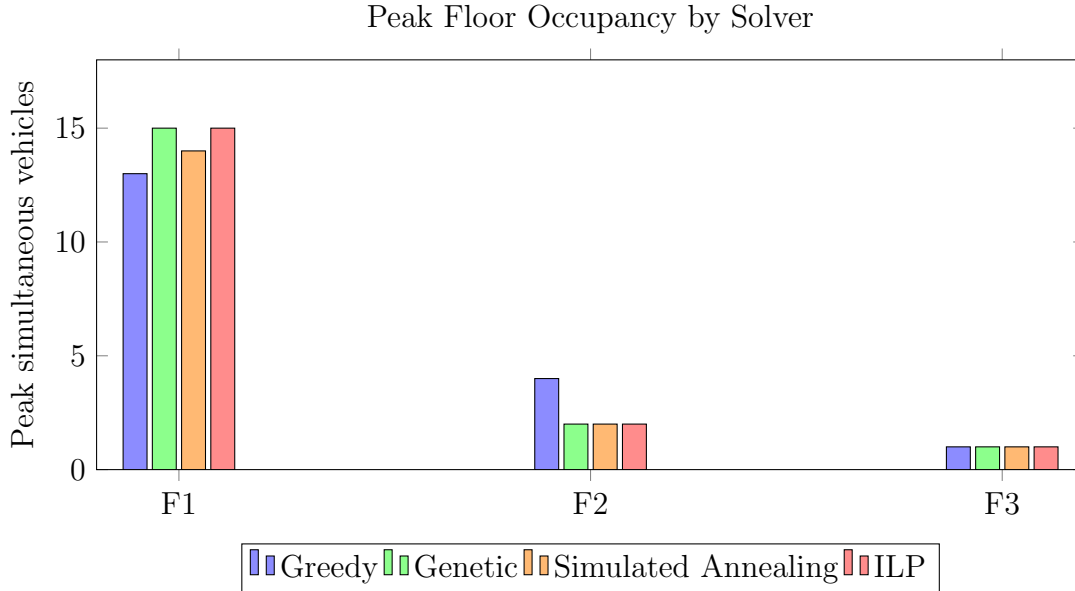


Figure 4: Peak simultaneous occupancy per floor. This metric reflects dynamic occupancy rather than cumulative assignments over the whole simulation.

10 Discussion

The results show that the ILP method obtains the best solution, with the lowest total cost and lowest average walking distance. This is expected because ILP is an offline exact method and has full knowledge of the vehicle arrival and departure times.

The greedy algorithm is the fastest method, but it produces the highest cost because it makes decisions locally without considering future arrivals.

The genetic algorithm improves the greedy solution, reducing the cost from 5254.5 to 4963.5. Simulated annealing performs even better in this experiment, reaching a cost of 4868.5.

The solution quality ranking is:

$$\text{ILP} < \text{Simulated Annealing} < \text{Genetic Algorithm} < \text{Greedy} \quad (9)$$

Since lower cost is better, this means ILP gives the best solution, followed by simulated annealing, then the genetic algorithm, then greedy.

All methods achieved:

- 100% EV satisfaction,
- 0% size violations,
- 1.7% unassigned vehicles.

The unassigned rate corresponds to approximately one vehicle out of 60. This can happen because of time overlap, size compatibility, or charger availability constraints.

11 Validation

The implementation was validated by running:

```
python main.py
python main.py --no-ga --no-sa --no-ilp
python -m py_compile main.py solver.py instance_gen.py evaluator.
py data_loader.py
```

The first command runs all solvers. The second command checks that the pipeline can run only the greedy baseline. The third command verifies that all Python files compile successfully.

12 Limitations

The main limitations of the current implementation are:

- ramp and elevator access capacity are not enforced as hard constraints,
- congestion is simplified,
- ILP is an offline benchmark and assumes future information is known,
- the parking geometry is synthetic,
- metaheuristic performance depends on parameter choices.

13 Future Work

Future improvements may include:

- adding hard ramp/elevator capacity constraints,
- modeling congestion more realistically,

- testing larger benchmark instances,
- generating more complex parking layouts,
- studying sensitivity to EV charger availability and demand intensity,
- producing animated or time-series occupancy visualizations,
- comparing additional metaheuristics such as tabu search or ant colony optimization.

14 Conclusion

This project implemented and compared several optimization approaches for a dynamic multi-level parking assignment problem. The implementation uses real dataset statistics to generate synthetic benchmark instances and evaluates four methods: greedy assignment, genetic algorithm, simulated annealing, and integer linear programming.

The implemented model respects the main constraints of the problem: vehicle assignment, space capacity over time, size compatibility, EV charging requirements, floor capacity, and dynamic arrivals and departures. The results show that heuristic methods improve over the greedy baseline, while ILP provides the best offline benchmark solution.